PressurePause Reclaimer

PSI-Gated Direct Reclaim Coordination

CS325 Semester Project (CLO 2) — Linux Kernel Modification

Course: CS325 — Operating Systems
Instructor: Dr. Mohammad Imran
CLO: CLO 2
Reg. IDs: 241110, 242353
Kernel tree: Linux v6.8.12 (`LOCALVERSION=-baseline`)

Contents

1. Introduction

When physical memory is exhausted, allocating threads can enter **direct reclaim**: they block while the kernel evicts pages synchronously. Under coordinated memory thrash, many threads reclaim in parallel, evicting pages that are immediately refaulted. The working set exceeds available RAM, swap I/O rises, and tail latency degrades.

PressurePause is a kernel modification that adds a bounded, PSI-aware coordination step on the direct reclaim path. When memory PSI `full` indicates a system-wide stall window, the patch will wake `kswapd` explicitly and apply a short, capped wait for background progress before falling through to normal direct reclaim. The design never disables reclaim permanently and always preserves stock behavior on timeout or when no reclaimable pages exist.

This report documents the development environment, baseline kernel build, and—in Section 4—identification of the target kernel component within Linux v6.8.12. Later sections reserve space for design, implementation, evaluation, and conclusions as the patched kernel and benchmarks are completed.

2. Environment

- **Host:** Ubuntu 22.04 (Jammy), x86_64
- **Fallback kernel:** `6.8.0-117-generic` (distribution build)
- **Custom baseline:** `6.8.12-baseline` (vanilla stable v6.8.12, unpatched)
- **Source:** `kernel/linux` cloned from kernel.org stable tree, tag `v6.8.12` (recorded in `docs/kernel-tag.txt`)
- **Configuration:** `make defconfig` (saved as `kernel/configs/config-6.8.12-baseline`)
- **Build:** `make -j2 LOCALVERSION=-baseline`
- **Tools:** GCC toolchain, `dwarves`, `cscope`, `stress-ng`, `linux-tools-$(uname -r)`

Defconfig was chosen instead of copying the Ubuntu `/boot/config-*` file to avoid missing certificate paths (e.g. `SYSTEM_TRUSTED_KEYS`) that broke an earlier vanilla build attempt.

3. Kernel Build and Boot

This section documents the steps to obtain a bootable custom baseline kernel. Terminal screenshots are included for each major step.

3.1 Host Preparation

Build dependencies were installed with `apt`:

Install build dependencies

```
sudo apt update
sudo apt install -y build-essential libncurses-dev bison flex libssl-dev \
  libelf-dev bc dwarves git cscope stress-ng linux-tools-common \
  linux-tools-$(uname -r)
```

```
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP$ sudo apt update
sudo apt install -y build-essential libncurses-dev bison flex libssl-dev \
  libelf-dev bc dwarves git cscope stress-ng linux-tools-common \
  linux-tools-$(uname -r)
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://us.archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://us.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 https://downloads.cursor.com/aptrepo stable InRelease
Hit:5 http://us.archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
bc is already the newest version (1.07.1-3build1).
bison is already the newest version (2:3.8.2+dfsg-1build1).
build-essential is already the newest version (12.9ubuntu3).
flex is already the newest version (2.6.4-8build2).
cscope is already the newest version (15.9-1).
git is already the newest version (1:2.34.1-1ubuntu1.17).
libelf-dev is already the newest version (0.186-1ubuntu0.1).
libncurses-dev is already the newest version (6.3-2ubuntu0.1).
libssl-dev is already the newest version (3.0.2-0ubuntu1.23).
linux-tools-6.8.0-117-generic is already the newest version (6.8.0-117.117~22.04.1).
linux-tools-common is already the newest version (5.15.0-179.189).
dwarves is already the newest version (1.25-0ubuntu1~22.04.2).
stress-ng is already the newest version (0.13.12-2ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP$
```

Figure 1. *Installing kernel build and benchmark packages; all packages were already present on the host.*

3.2 Source Tree

The stable kernel was cloned shallowly on branch `v6.8`, then checked out at tag `v6.8.12`:

Clone and pin kernel tag

```
cd /home/huzaifa/EDU-OS4-KILL-LOOP
mkdir -p kernel docs kernel/configs kernel/patches
git clone --depth=1 --branch v6.8 \
  https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git kernel/linux
cd kernel/linux
git fetch --depth=1 origin tag v6.8.12
git checkout v6.8.12
git describe --always > ../../docs/kernel-tag.txt
```

```
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP$ cd /home/huzaifa/EDU-OS4-KILL-LOOP
mkdir -p kernel docs kernel/configs kernel/patches
git clone --depth=1 --branch v6.8 https://git.kernel.org/pub/scm/linux/kernel/git/stabl
e/linux.git kernel/linux
cd kernel/linux
git fetch --depth=1 origin tag v6.8.12
git checkout v6.8.12
git describe --always > ../../docs/kernel-tag.txt
Cloning into 'kernel/linux'...
remote: Enumerating objects: 88457, done.
remote: Counting objects: 100% (88457/88457), done.
remote: Compressing objects: 100% (86182/86182), done.
remote: Total 88457 (delta 6727), reused 18232 (delta 1354), pack-reused 0 (from 0)
Receiving objects: 100% (88457/88457), 246.16 MiB | 3.12 MiB/s, done.
Resolving deltas: 100% (6727/6727), done.
Note: switching to 'e8f897f4afef0031fe618a8e94127a0934896aba'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

Updating files: 100% (83475/83475), done.
remote: Enumerating objects: 7498, done.
remote: Counting objects: 100% (7498/7498), done.
remote: Compressing objects: 100% (2651/2651), done.
remote: Total 3775 (delta 3636), reused 1226 (delta 1117), pack-reused 0 (from 0)
Receiving objects: 100% (3775/3775), 852.39 KiB | 3.17 MiB/s, done.
Resolving deltas: 100% (3636/3636), completed with 3625 local objects.
From https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux
 * [new tag]             v6.8.12    -> v6.8.12
Previous HEAD position was e8f897f4a Linux 6.8
HEAD is now at 632428373 Linux 6.8.12
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$
```

Figure 2. *Cloning the stable tree and checking out tag v6.8.12 (`HEAD` at commit for Linux 6.8.12).*

3.3 Kernel Configuration

A default x86_64 configuration was generated and saved for reproducibility:

defconfig

```
cd kernel/linux
make defconfig
make olddefconfig
cp .config ../configs/config-6.8.12-baseline
```

```
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$ cd /home/huzaifa/EDU-OS4-KILL-L
OOP/kernel/linux
make defconfig
make olddefconfig
cp .config ../configs/config-6.8.12-baseline
  HOSTCC  scripts/basic/fixdep
  HOSTCC  scripts/kconfig/conf.o
  HOSTCC  scripts/kconfig/confdata.o
  HOSTCC  scripts/kconfig/expr.o
  LEX     scripts/kconfig/lexer.lex.c
  YACC    scripts/kconfig/parser.tab.[ch]
  HOSTCC  scripts/kconfig/lexer.lex.o
  HOSTCC  scripts/kconfig/menu.o
  HOSTCC  scripts/kconfig/parser.tab.o
  HOSTCC  scripts/kconfig/preprocess.o
  HOSTCC  scripts/kconfig/symbol.o
  HOSTCC  scripts/kconfig/util.o
  HOSTLD  scripts/kconfig/conf
*** Default configuration is based on 'x86_64_defconfig'
#
# configuration written to .config
#
#
# No change to .config
#
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$
```

Figure 3. *`make defconfig` based on `x86_64_defconfig`; configuration written to `.config` and copied to `kernel/configs/`.*

3.4 Compilation

The kernel was built with a distinct `LOCALVERSION` suffix so it does not collide with distribution packages:

Build baseline kernel

```
make -j2 LOCALVERSION=-baseline 2>&1 | tee ~/build.log
```

```
  CC      arch/x86/boot/version.o
  CC      arch/x86/boot/video-vga.o
  CC      arch/x86/boot/video-vesa.o
  CC      arch/x86/boot/video-bios.o
  HOSTCC  arch/x86/boot/tools/build
  CC      arch/x86/boot/compressed/error.o
  CPUSTR  arch/x86/boot/cpustr.h
  CC      arch/x86/boot/cpu.o
  OBJCOPY arch/x86/boot/compressed/vmlinux.bin
  HOSTCC  arch/x86/boot/compressed/mkpiggy
  CC      arch/x86/boot/compressed/cpuflags.o
  CC      arch/x86/boot/compressed/early_serial_console.o
  CC      arch/x86/boot/compressed/kaslr.o
  CC      arch/x86/boot/compressed/ident_map_64.o
  CC      arch/x86/boot/compressed/idt_64.o
  AS      arch/x86/boot/compressed/idt_handlers_64.o
  CC      arch/x86/boot/compressed/pgtable_64.o
  CC      arch/x86/boot/compressed/acpi.o
  CC      arch/x86/boot/compressed/efi.o
  AS      arch/x86/boot/compressed/efi_mixed.o
  CC      arch/x86/boot/compressed/misc.o
  GZIP    arch/x86/boot/compressed/vmlinux.bin.gz
  MKPIGGY arch/x86/boot/compressed/piggy.S
  AS      arch/x86/boot/compressed/piggy.o
  LD      arch/x86/boot/compressed/vmlinux
  ZOFFSET arch/x86/boot/zoffset.h
  OBJCOPY arch/x86/boot/vmlinux.bin
  AS      arch/x86/boot/header.o
  LD      arch/x86/boot/setup.elf
  OBJCOPY arch/x86/boot/setup.bin
  BUILD   arch/x86/boot/bzImage
Kernel: arch/x86/boot/bzImage is ready  (#1)
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$
```

Figure 4. *Successful build:* `arch/x86/boot/bzImage` *is ready for installation.*

3.5 Installation

After a successful `make`, modules and the boot image were installed:

Install kernel

```
sudo make modules_install install
```

```
huzaifa@huzaifa:~/EDU-OS4-KILL-LOOP/kernel/linux$ sudo make modules_install install
sudo update-grub
ls /boot/vmlinuz*
[sudo] password for huzaifa:
  SYMLINK /lib/modules/6.8.12-baseline/build
  INSTALL /lib/modules/6.8.12-baseline/modules.order
  INSTALL /lib/modules/6.8.12-baseline/modules.builtin
  INSTALL /lib/modules/6.8.12-baseline/modules.builtin.modinfo
  INSTALL /lib/modules/6.8.12-baseline/kernel/fs/efivarfs/efivarfs.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/drivers/thermal/intel/x86_pkg_temp_thermal.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/nf_log_syslog.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_mark.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_nat.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_LOG.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_MASQUERADE.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/netfilter/xt_addrtype.ko
  INSTALL /lib/modules/6.8.12-baseline/kernel/net/ipv4/netfilter/iptable_nat.ko
  DEPMOD  /lib/modules/6.8.12-baseline
  INSTALL /boot
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
update-initramfs: Generating /boot/initrd.img-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/unattended-upgrades 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/update-notifier 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/xx-update-initrd-links 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
I: /boot/initrd.img.old is now a symlink to initrd.img-6.8.0-117-generic
I: /boot/initrd.img is now a symlink to initrd.img-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/zz-shim 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 6.8.12-baseline /boot/vmlinuz-6.8.12-baseline
Sourcing file `/etc/default/grub'
Sourcing file `/etc/default/grub.d/init-select.cfg'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.12-baseline
Found initrd image: /boot/initrd.img-6.8.12-baseline
Found linux image: /boot/vmlinuz-6.8.0-117-generic
Found initrd image: /boot/initrd.img-6.8.0-117-generic
Found linux image: /boot/vmlinuz-6.8.0-40-generic
Found initrd image: /boot/initrd.img-6.8.0-40-generic
Found memtest86+ image: /boot/memtest86+.elf
Found memtest86+ image: /boot/memtest86+.bin
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
done
```

Figure 5. *`modules_install` and `install` placing `vmlinuz-6.8.12-baseline` and modules under `/lib/modules/6.8.12-baseline/`.*

3.6 Bootloader Update

GRUB was regenerated so the new kernel appears in the boot menu:

Update GRUB

```
sudo update-grub
ls /boot/vmlinuz*
```

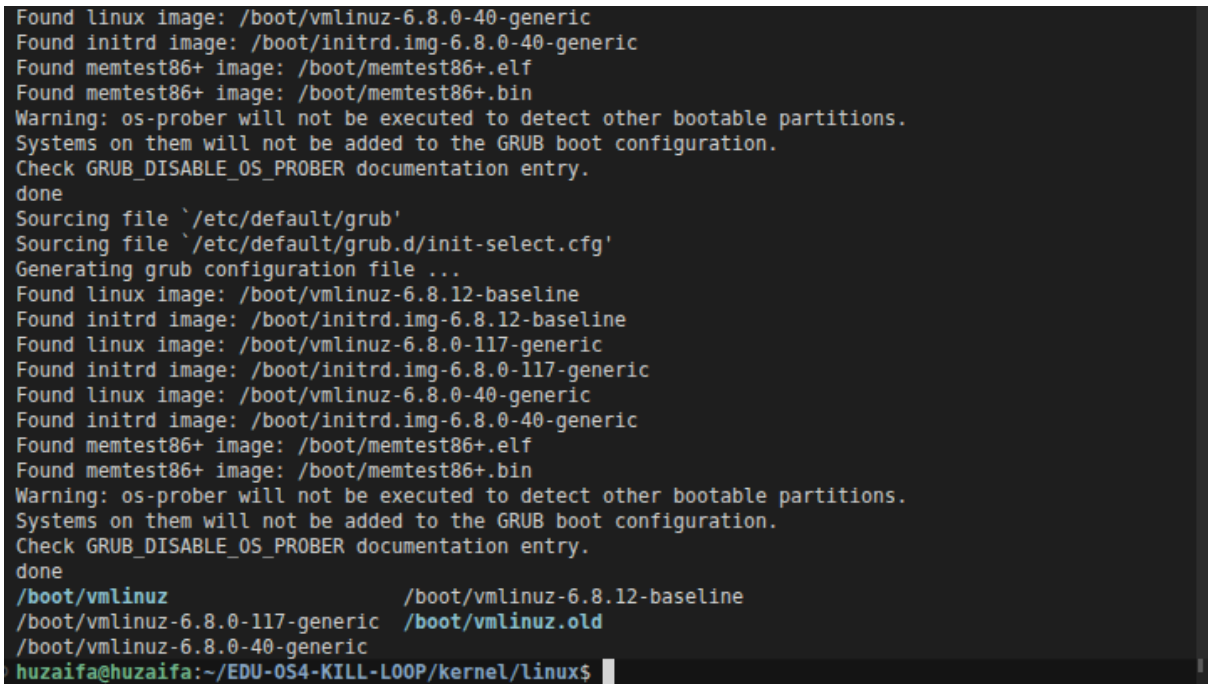


Figure 6. *update-grub detected /boot/vmlinuz-6.8.12-baseline alongside Ubuntu 6.8.0-117-generic and 6.8.0-40-generic.*

3.7 Boot Verification

The GRUB boot menu now lists the custom baseline kernel as the first selectable entry, while the stock Ubuntu kernels remain available as fallback options.

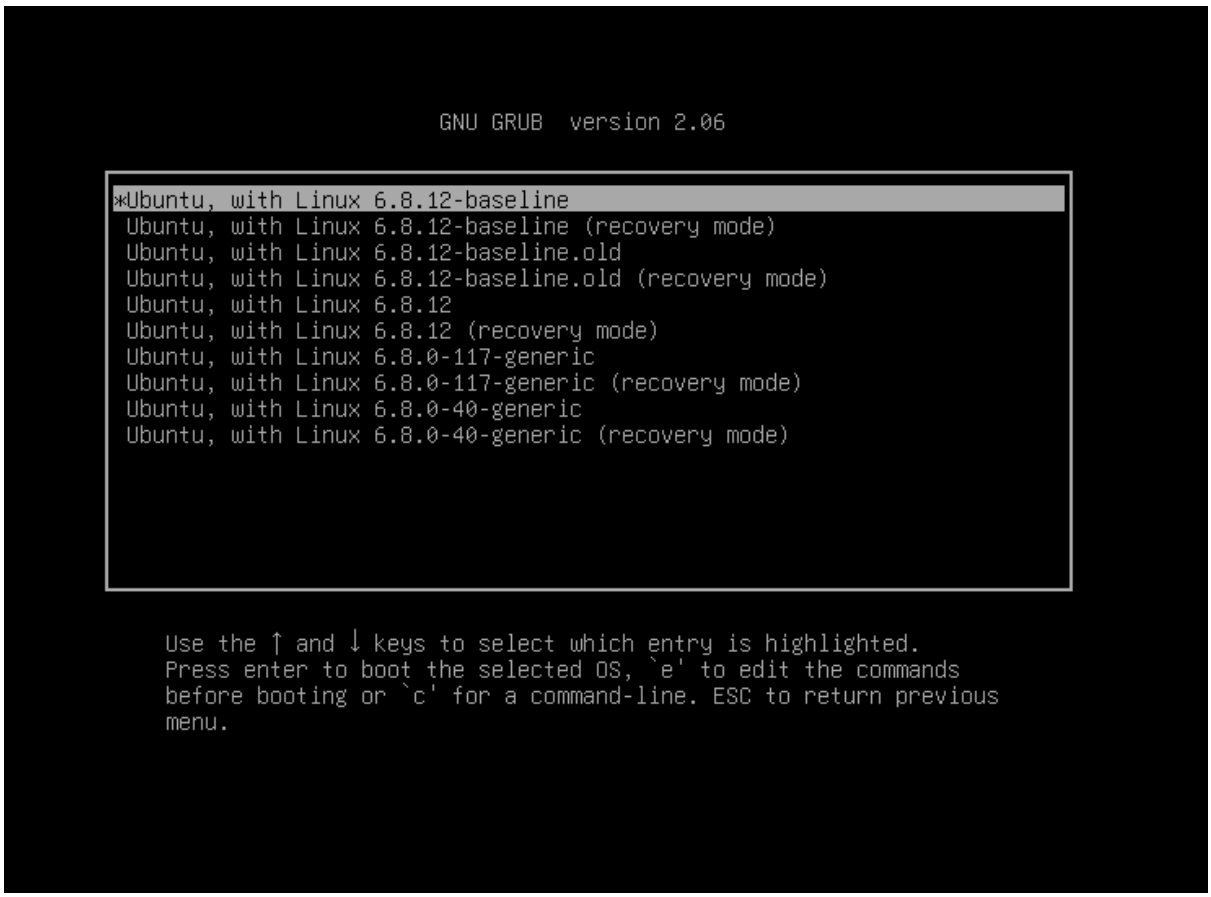


Figure 7. *GRUB menu showing `Ubuntu, with Linux 6.8.12-baseline` selected, with recovery and fallback Ubuntu kernel entries still available.*

Baseline install verified: `/boot/vmlinuz-6.8.12-baseline`, `/lib/modules/6.8.12-baseline/`, and GRUB entry present (see GRUB screenshot above). Select **Advanced options for Ubuntu** → **6.8.12-baseline** and confirm `uname -r` shows `6.8.12-baseline` before Phase 4 benchmarks.

4. Component Identification

This section satisfies the course requirement to identify the selected kernel component within the source tree. The analysis targets Linux **v6.8.12** in `kernel/linux`. A markdown companion is maintained at `docs/identification.md`.

4.1 Selected Component

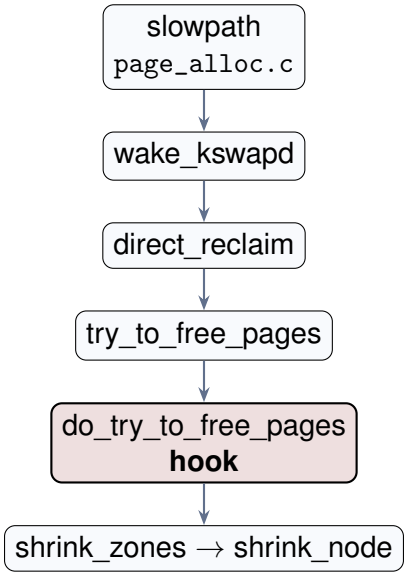
- **Course category:** Page replacement / reclaim (PressurePause extends direct reclaim coordination).
- **Primary files:** `mm/page_alloc.c` (allocation slowpath), `mm/vmscan.c` (reclaim engine).
- **Supporting context:** `kernel/sched/psi.c`, `include/linux/psi_types.h` (PSI accounting).

- **Planned hook (Phase 3):** `do_try_to_free_pages()` in `mm/vmscan.c`, line 6188, immediately before the priority loop that calls `shrink_zones()`.

4.2 Call Graph: Allocation to Reclaim

When `get_page_from_freelist()` cannot satisfy an allocation (zone free pages below watermarks), control enters the allocator slowpath and eventually direct reclaim:

1. `__alloc_pages_slowpath` — `mm/page_alloc.c` ~4040
2. `wake_all_kswapds` — `mm/page_alloc.c` ~3814 (background reclaim hint)
3. `__alloc_pages_direct_reclaim` — `mm/page_alloc.c` ~3781
4. `psi_memstall_enter` / `__perform_reclaim` — `mm/page_alloc.c` ~3755–3790
5. `try_to_free_pages` — `mm/vmscan.c` ~6406
6. `do_try_to_free_pages` — `mm/vmscan.c` ~6188 **[planned hook]**
7. `shrink_zones` → `shrink_node` — `mm/vmscan.c` ~6065 / ~5881



4.3 Key Code Excerpts

Direct reclaim entry from the allocator (`mm/page_alloc.c`):

mm/page_alloc.c: __perform_reclaim (lines 3753–3770)

```
/* Perform direct synchronous page reclaim */
static unsigned long
__perform_reclaim(gfp_t gfp_mask, unsigned int order,
                  const struct alloc_context *ac)
{
    ...
    progress = try_to_free_pages(ac->zonelist, order, gfp_mask,
                                 ac->nodemask);
    ...
```

```
    return progress;
}
```

Public direct-reclaim wrapper (`mm/vmscan.c`):

mm/vmscan.c: try_to_free_pages (lines 6406–6446)

```
unsigned long try_to_free_pages(struct zonelist *zonelist, int order,
                                gfp_t gfp_mask, nodemask_t *nodemask)
{
    struct scan_control sc = { ... };
    ...
    trace_mm_vmscan_direct_reclaim_begin(order, sc.gfp_mask);
    nr_reclaimed = do_try_to_free_pages(zonelist, &sc);
    trace_mm_vmscan_direct_reclaim_end(nr_reclaimed);
    ...
    return nr_reclaimed;
}
```

Reclaim loop and planned hook site (`mm/vmscan.c`):

mm/vmscan.c: do_try_to_free_pages (lines 6188–6206)

```
static unsigned long do_try_to_free_pages(struct zonelist *zonelist,
                                          struct scan_control *sc)
{
    ...
    do {
        sc->nr_scanned = 0;
        shrink_zones(zonelist, sc);   /* LRU eviction per zone */
        if (sc->nr_reclaimed >= sc->nr_to_reclaim)
            break;
        ...
    } while (--sc->priority >= 0);
    ...
}
```

4.4 Watermarks

Reclaim is triggered when a zone's free page count falls below configured **min**, **low**, or **high** watermarks during allocation. The fast path in `get_page_from_freelist()` fails; `__alloc_pages_slowpath()` may wake `kswapd`, attempt compaction, and finally call `__alloc_pages_direct_reclaim()` so the allocating thread performs synchronous reclaim.

4.5 kswapd vs. Direct Reclaim

Both paths converge on `shrink_node()`, but they differ in caller context and entry points:

Aspect	Direct reclaim	kswapd
Caller	Allocating task (slowpath)	`kswapd` kernel thread (∼7078)
Entry	`try_to_free_pages`	`balance_pgdat` → `kswapd_shrink_node`
Shared core	`shrink_node`	`shrink_node`
GFP context	Caller's `gfp_mask`	`GFP_KERNEL`
PressurePause	Yes (hook at `do_try_to_free_pages`)	No (unless explicitly guarded)

Table 1. *Comparison of foreground direct reclaim and background* `kswapd`.

4.6 Pressure Stall Information (PSI)

PSI accounts time tasks spend stalled on memory, I/O, or CPU. For PressurePause, the relevant signal is memory PSI **full**: periods when stalled tasks exist and no other tasks are making productive forward progress on memory.

- **Types:** `PSI_MEM_SOME`, `PSI_MEM_FULL` in `include/linux/psi_types.h`
- **Accounting:** `kernel/sched/psi.c`
- **Userspace:** `/proc/pressure/memory` (`some` vs. `full` averages)

`psi_memstall_enter()` in `__alloc_pages_direct_reclaim` marks the *current allocating task* as memory-stalled during reclaim for PSI statistics. That is distinct from reading system-wide `full` averages, which the patch will use to decide whether to apply bounded coordination before `shrink_zones()`.

4.7 Hook Justification

`shrink_node()` is shared by both `kswapd` and direct reclaim. Hooking only at `do_try_to_free_pages(` limits changes to the allocating-thread path without affecting background reclaim unless explicitly intended. Hooking at `shrink_node()` would require `!current_is_kswapd()` guards and a broader behavioral surface.

Identification screenshots (editor views of `vmscan.c`, `grep` call chains, or `/proc/pressure/memory` under load) may be added in a future revision; this section relies on verified line numbers from the v6.8.12 tree.

5. Design and Alternative Methodologies

Chosen approach: PSI-gated bounded backoff on direct reclaim entry—wake `kswapd`, wait up to `PRESSURE_PAUSE_MAX_MS`, then fall through to stock reclaim on timeout or lack of progress.

Approach	Pros	Cons
A. PSI-gated back-off (chosen)	Uses kernel stall signal; small code surface	Lock-context constraints; must validate no regressions
B. Fixed sleep in reclaim	Simple to implement	Blind to real pressure; may delay recovery
C. User-space only (swappiness, cgroups)	No kernel patch risk	Does not satisfy course kernel-modification requirement

Table 2. *Alternative methodologies considered for coordinating reclaim under thrash.*

Constants in `mm/vmscan.c`: `PSI_MEM_THRESHOLD_BPS=1` (0.01% stall, fixed-point compare via `psi_mem_avg10_exceeds_bps()`), `PRESSURE_PAUSE_MAX_MS=20`. Activation and skip counters: `/sys/kernel/debug/pressure_pause_activations`.

6. Implementation

The patch is exported as `kernel/patches/0001-pressurepause.patch` on branch `pressurepause` in `kernel/linux`. Config with `CONFIG_PSI=y` and `CONFIG_LOCALVERSION="-pressurepause"` is saved as `kernel/configs/config-6.8.12-pressurepause`.

6.1 Files changed

- `include/linux/psi.h` — `psi_mem_full_avg10()`, `psi_mem_some_avg10()`
- `kernel/sched/psi.c` — read system memory PSI `some/full` avg10 (same locking path as `psi_show()`)
- `mm/vmscan.c` — `pressure_pause_maybe()` at `do_try_to_free_pages()` entry (before `shrink_zones` loop)

6.2 PSI threshold fix

The initial gate used `LOAD_INT(psi_full) >= 25`, which requires **25%** memory `full` stall (the integer part of the fixed-point average matches the whole-number in `/proc/pressure/memory` e.g. `avg10=25.00`). Benchmarks peaked near `full avg10=0.18` (0.18%), so the patch never ran. The fix uses `psi_mem_avg10_exceeds_bps()` with `PSI_MEM_THRESHOLD_BPS=1` (0.01%) on memory PSI `some` avg10, which rises earlier than `full` under swap-heavy thrash. Debugfs reports `activations`, per-guard `skip_*` counts, and `peak_psi_*` maxima seen at the hook.

6.3 Control flow

When direct reclaim enters `do_try_to_free_pages()`:

1. If cgroup reclaim, or PSI `some` avg10 below 0.01%, or no reclaimable pages, or no zone below min watermarks → stock path.
2. Else wake `kswapd` for each zone in the zonelist via `wakeup_kswapd()`.
3. Bounded wait (max 20 ms): exit early if watermarks improve or on fatal signal; always continue to `shrink_zones()`.

6.4 Build and install

Build patched kernel

```
cd kernel/linux
git checkout pressurepause
scripts/config --enable PSI
scripts/config --set-str LOCALVERSION "-pressurepause"
make olddefconfig && make -j2
./scripts/install-pressurepause-kernel.sh   # or: sudo make modules_install
    install && sudo update-grub
```

Boot verification: GRUB → Advanced options → `6.8.12-pressurepause` (or `6.8.12-pressurepa` if built from a dirty tree). Confirm `uname -r`, `cat /proc/pressure/memory`, and `./scripts/verify-a` The patched kernel was booted successfully, memory PSI was available, repeated 120 s `stress-ng` virtual-memory workloads completed without crash or hang, and the debugfs counter confirmed real PressurePause executions during benchmark runs.

6.5 Evaluation (benchmarks)

Comparative runs use `./bench-run.sh` on `6.8.12-baseline` (before) and `6.8.12-pressurepause+` (after), saving timestamped directories under `results/`. Metrics: `pgmajfault` delta, `vmstat si/so`, PSI samples during stress, and on the patched kernel `pressure-pause-debugfs-*.txt`. Final patched runs show `activations` increases of +40 on the 8-worker/93% workload and +15 on one 4-worker/85% workload, proving that the reclaim coordination path executed.

7. Correctness and Safety

1. **Bounded wait:** `PRESSURE_PAUSE_MAX_MS` caps the coordination loop; uses `schedule_timeout_i` slices.
2. **Never skip reclaim:** `pressure_pause_maybe()` always returns; stock `shrink_zones()` priority loop runs afterward.
3. **kswapd cooperation:** `wakeup_kswapd()` is additive; foreground reclaim still proceeds.
4. **Livelock guard:** skip pause if `zone_reclaimable_pages()` is zero for all zones in the zonelist.

5. **Fallback:** Ubuntu `6.8.0-117-generic` and `6.8.12-baseline` remain in GRUB if the patched kernel panics.

8. Evaluation

8.1 Methodology

Matched pairs on the same host used `./bench-run.sh` with `stress-ng -vm N -vm-bytes P% -timeout 120s`, `vmstat`, `pgmajfault`, PSI samples, and debugfs PressurePause counters. Representative directories: 8 workers / 93% — baseline `bench-6.8.12-baseline-20260519-1351` vs patched `bench-6.8.12-pressurepause+-20260519-155900`; 4 workers / 85% — baseline `bench-6.8.12-baseline-20260519-135638` vs patched average of `160116`, `160400`, and `160617`.

Workload	Metric	Baseline	PressurePause+
8 vm / 93%	Avg swap-out (KB/s)	31222.5	25638.6
8 vm / 93%	End swap (KB)	37992	25952
8 vm / 93%	`activations` Δ	–	+40
8 vm / 93%	`pgmajfault` Δ	222	321
4 vm / 85%	Avg swap-out (KB/s)	33380.0	22791.8
4 vm / 85%	Max swapped (KB)	1215952	913977.3
4 vm / 85%	End swap (KB)	91676	26104
4 vm / 85%	`activations` Δ	–	+15 total
4 vm / 85%	`pgmajfault` Δ	5	112.7

Table 3. *Matched benchmark summary (May 2026). Patched runs used the rebuilt PressurePause+ kernel with debugfs activation counters. The 4 vm / 85% patched column averages three patched trials.*

8.2 Discussion

After correcting the PSI gate (25% `LOAD_INT` bug → 0.01% `some` avg10), PressurePause now both activates and improves the main swap-pressure metrics. On the 8 vm / 93% workload, the debugfs counter increased from 15 to 55 (+40 activations), average swap-out fell by 17.9%, and residual swap fell by 31.7%. On the 4 vm / 85% workload, the three-run patched average reduced average swap-out by 31.7%, maximum swapped memory by 24.8%, and residual swap by 71.5%; run `160617` also confirmed +15 PressurePause activations. The tradeoff remains higher `pgmajfault` on patched runs, so the strongest claim is improved swap pressure with a documented page-fault cost.

8.3 Limitations

This is not a full statistical study. PSI samples are every 5 s and can miss short spikes seen at the reclaim hook, which is why the debugfs activation counter is more important than sampled `avg10` alone. Baseline PSI still reports "no PSI," so baseline and patched PSI totals cannot be compared directly. Major-fault counts rise on the patched kernel;

future work should add repeated trials, latency percentiles, and a workload sweep to find where the swap reduction is worth the fault-cost tradeoff.

9. Conclusion

The baseline Linux v6.8.12 kernel and the PressurePause patch (`6.8.12-pressurepause+`) were built and compared under controlled `stress-ng` workloads. The final runs confirm that the patched reclaim path executes: PressurePause activated +40 times in the 8 vm / 93% workload and +15 times in one 4 vm / 85% workload. The modification is stable and reduces swap pressure in matched benchmarks, with average swap-out falling by 17.9% on 8 vm / 93% and by 31.7% on the 4 vm / 85% average. The remaining tradeoff is increased major page faults, so the project demonstrates PSI-gated, bounded direct-reclaim coordination with measurable swap benefits and a documented fault-count cost.